

Module : Kotlin (M202)

TP N° 12

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVM 101 - DEVM 102 Niveau : 2ème année

Exercice 1 :

1. Crée une fonction **findMax** qui prend deux paramètres :
 - Une liste d'entiers.
 - Une fonction lambda qui prend deux entiers et retourne celui qui est le plus grand.
2. Utilise la fonction **findMax** pour comparer les éléments de la liste et trouver le plus grand nombre.
 - Appelle la fonction avec une liste de ton choix et affiche le résultat.

Exercice 2 :

1. Crée une fonction **countElements** qui prend deux paramètres :
 - Une liste d'entiers.
 - Une fonction **lambda** qui prend un entier et retourne un booléen (true si l'élément respecte la condition, sinon false).
 - La fonction doit parcourir la liste et compter les éléments pour lesquels la lambda retourne true.
2. Appelle la fonction avec une liste d'entiers et différentes lambdas :
 - Une lambda pour compter les nombres pairs.
 - Une lambda pour compter les nombres supérieurs à 10.

Exercice 3 :

1. Crée une fonction **calculate** qui prend trois paramètres :

- Deux nombres entiers.
 - Une fonction anonyme qui prend deux entiers en paramètres et retourne un entier.
2. Utilise la fonction `calculate` pour effectuer différentes opérations mathématiques (addition, soustraction, multiplication, division) en passant des fonctions anonymes directement lors de l'appel.
 3. Affiche les résultats pour plusieurs opérations.

Exercice 4 :

1. Créez une fonction anonyme qui prend un entier et retourne `true` si le nombre est positif, et `false` sinon.
2. Utilisez cette fonction anonyme pour tester plusieurs nombres.
3. Affichez si chaque nombre est positif ou négatif.

Exercice 5 :

1. Créer une fonction **`readNumber`** qui demande à l'utilisateur d'entrer un nombre entier.
2. Utilise un bloc `try-catch` pour gérer le cas où l'utilisateur entre une valeur non entière (par exemple, du texte ou un nombre à virgule).
3. Si l'utilisateur entre un nombre entier valide, affiche le nombre avec un message.
4. Si une exception est levée (par exemple, parce que l'utilisateur a entré une valeur invalide), affiche un message d'erreur approprié et demande à l'utilisateur d'entrer un nombre valide.

Exercice 6 :

1. Crée une fonction **`convertToIntList`** qui prend une liste de chaînes (`List<String>`) comme paramètre.
2. Dans cette fonction, utilise un bloc **`try-catch`** pour tenter de convertir chaque chaîne en entier.

3. Si la conversion réussit, ajoute le nombre à une nouvelle liste.
4. Si une chaîne ne peut pas être convertie (par exemple, si elle contient du texte), attrape l'exception et affiche un message d'erreur, puis passe à la chaîne suivante.
5. Retourne la liste des entiers convertis.